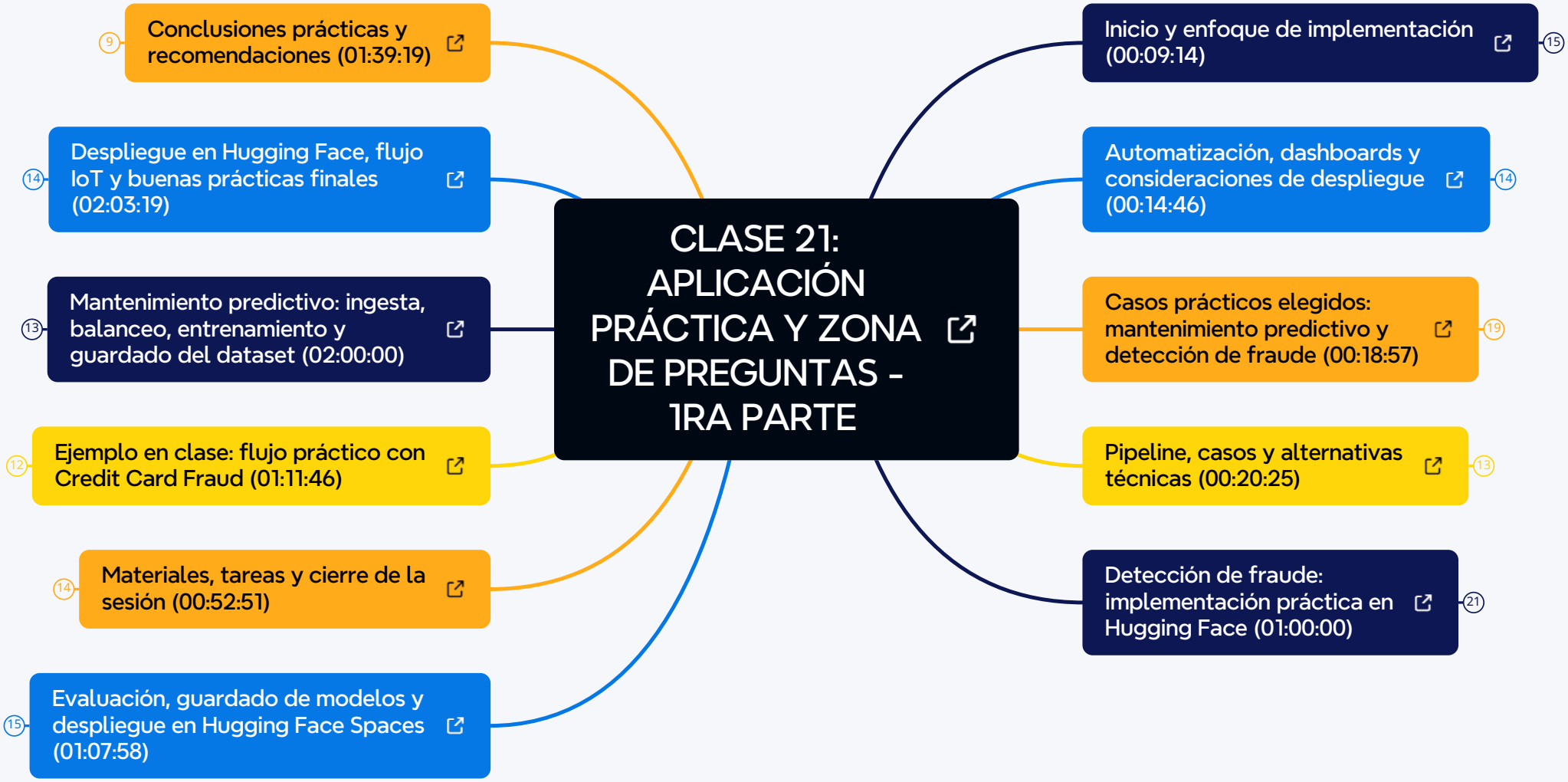
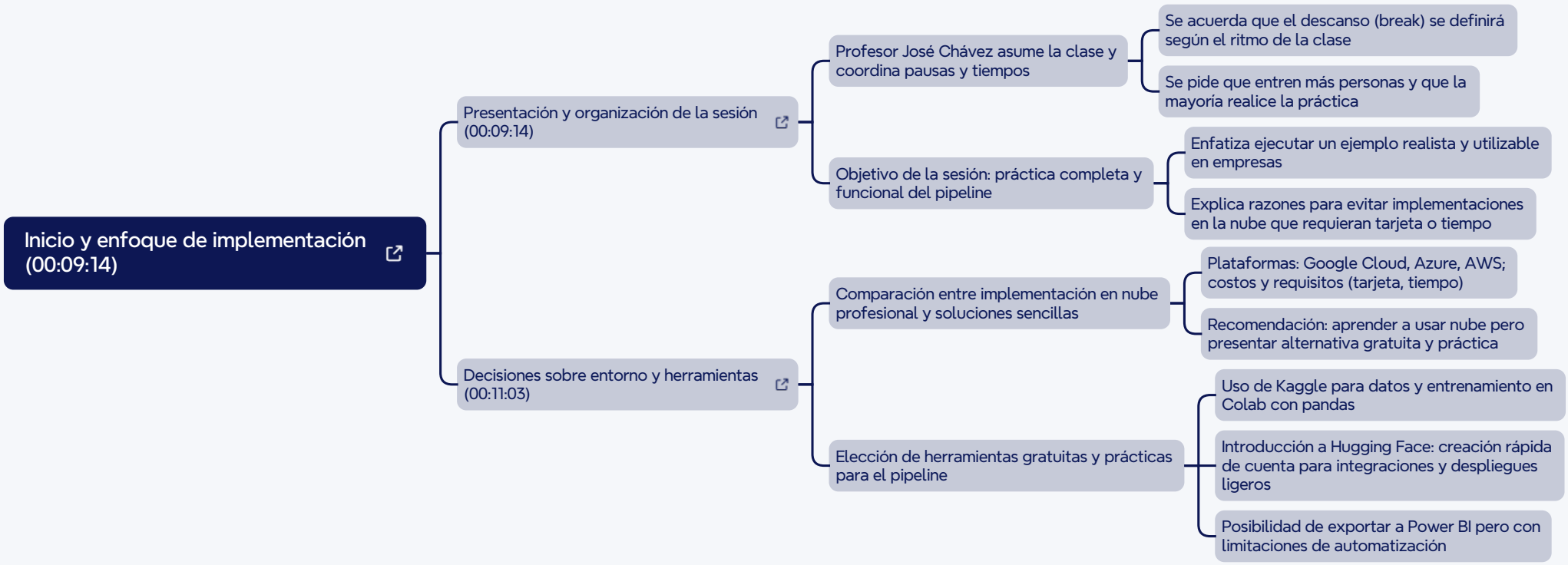
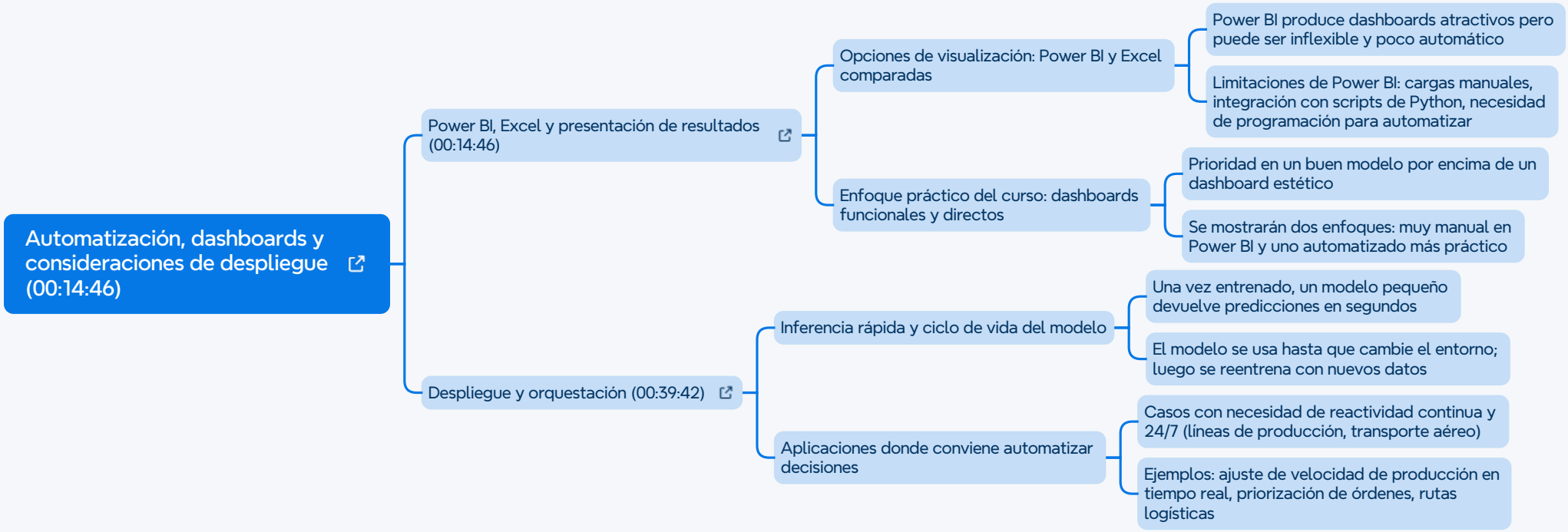
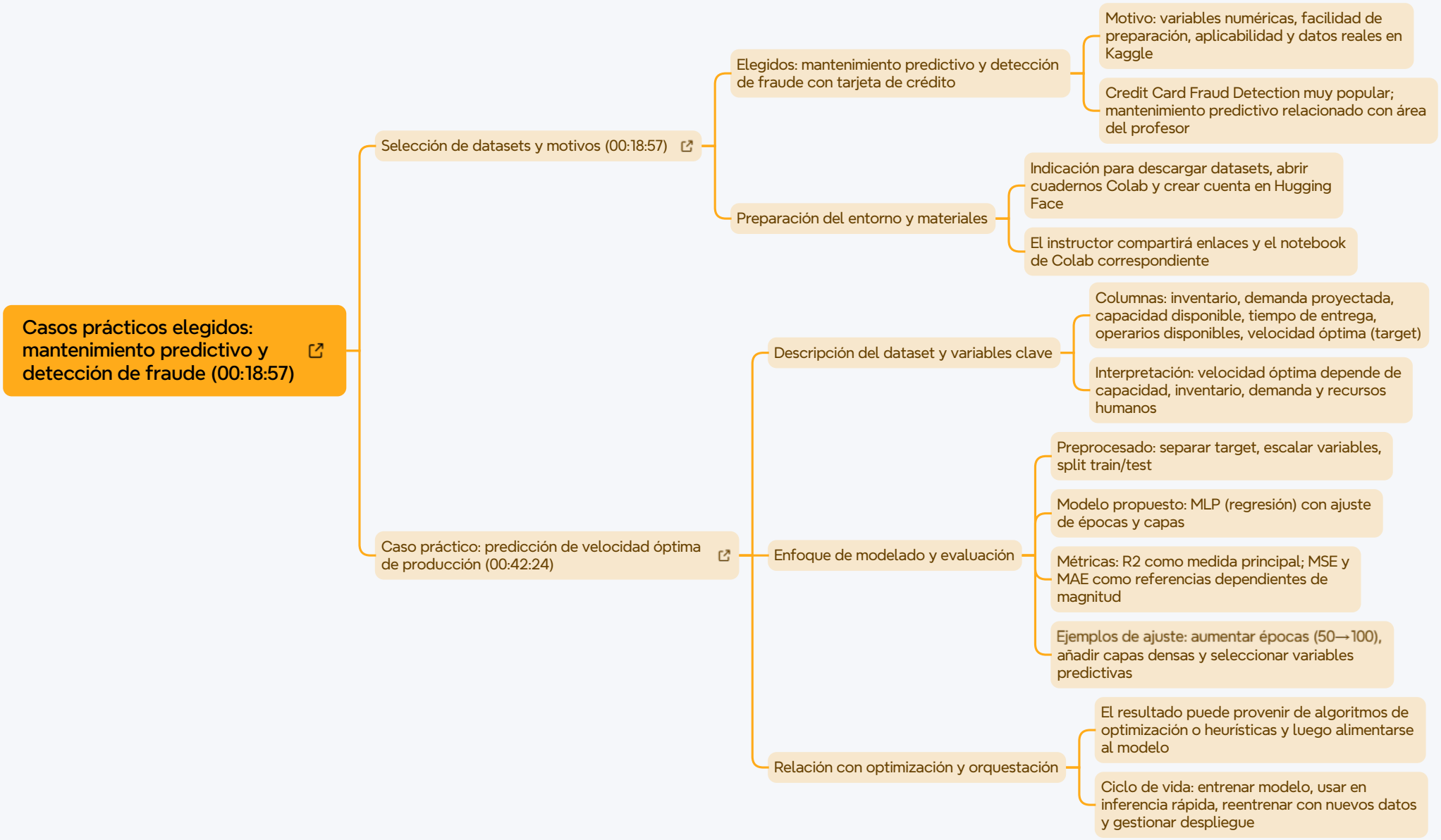
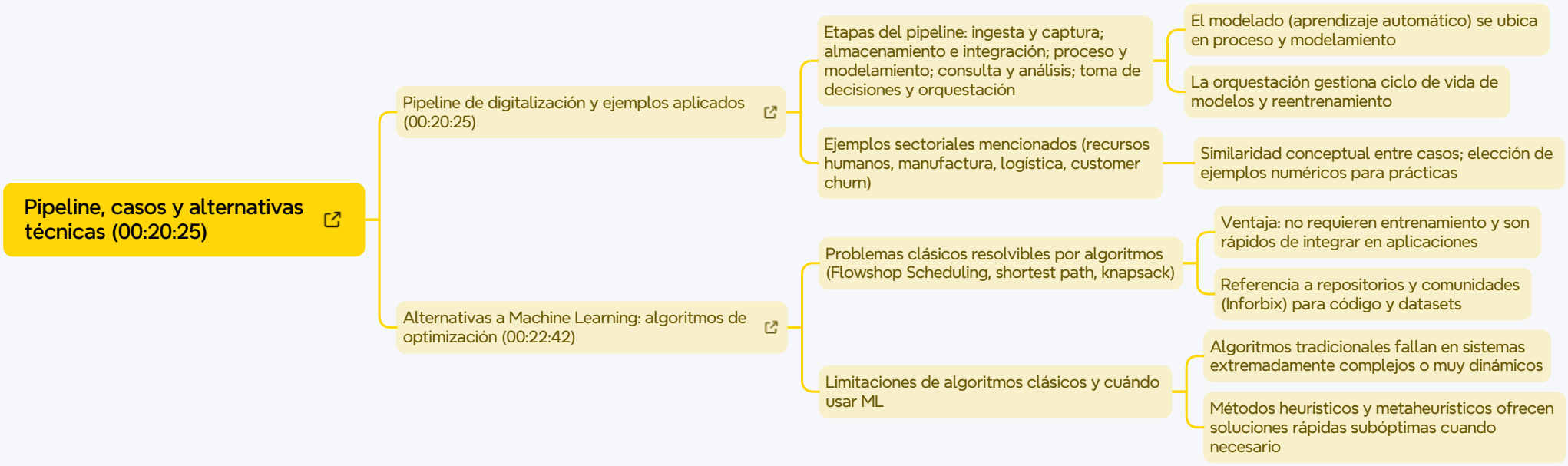
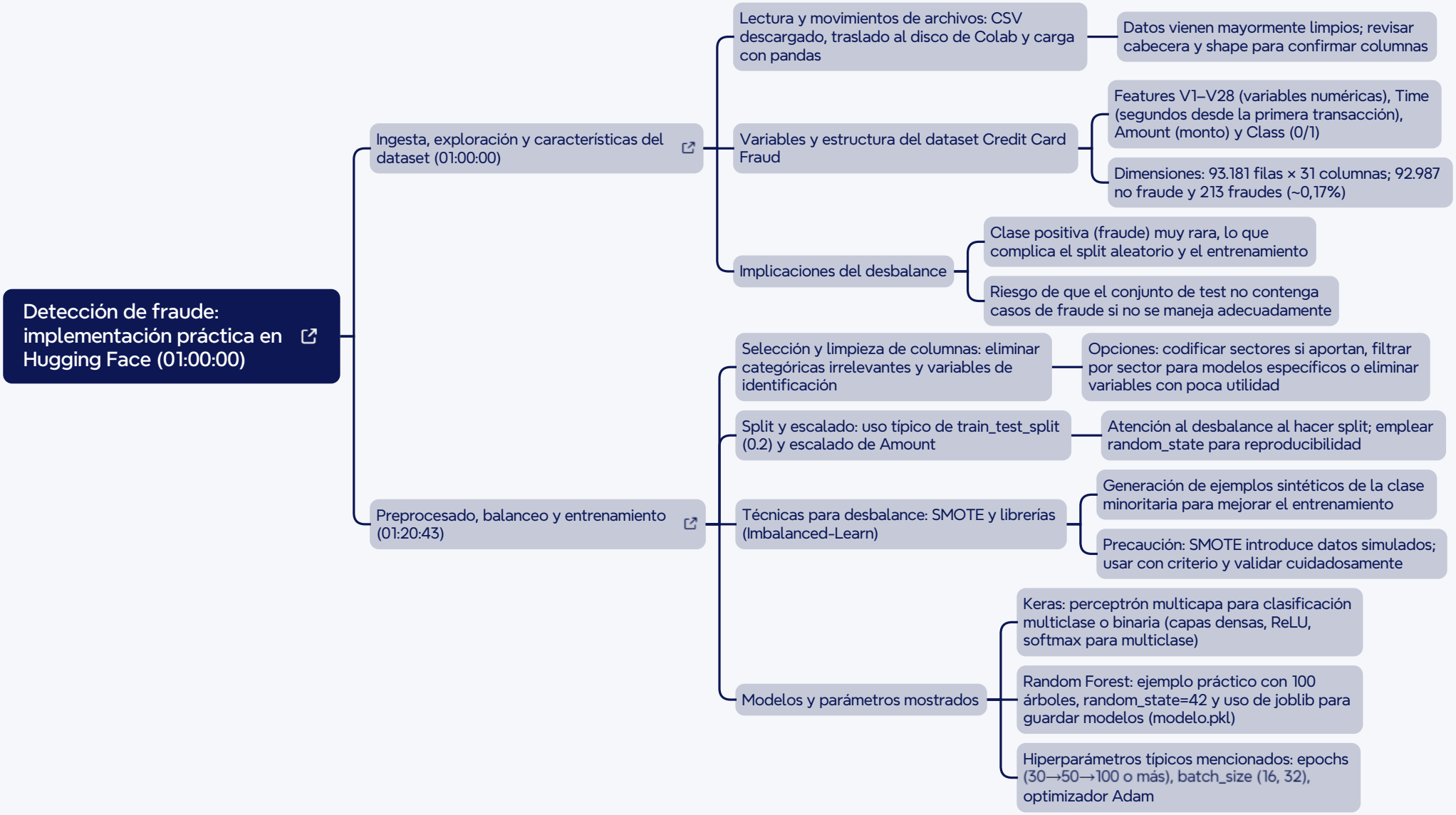

CLASE 21: APLICACIÓN PRÁCTICA Y ZONA DE PREGUNTAS - 1RA PARTE
Conclusiones prácticas y recomendaciones (01:39:19)
Despliegue en Hugging Face, flujo IoT y buenas prácticas finales (02:03:19)
Mantenimiento predictivo: ingesta, balanceo, entrenamiento y guardado del dataset (02:00:00)
Ejemplo en clase: flujo práctico con Credit Card Fraud (01:11:46)
Materiales, tareas y cierre de la sesión (00:52:51)
Evaluación, guardado de modelos y despliegue en Hugging Face Spaces (01:07:58)
Inicio y enfoque de implementación (00:09:14)
Automatización, dashboards y consideraciones de despliegue (00:14:46)
Casos prácticos elegidos: mantenimiento predictivo y detección de fraude (00:18:57)
Pipeline, casos y alternativas técnicas (00:20:25)
Detección de fraude: implementación práctica en Hugging Face (01:00:00)

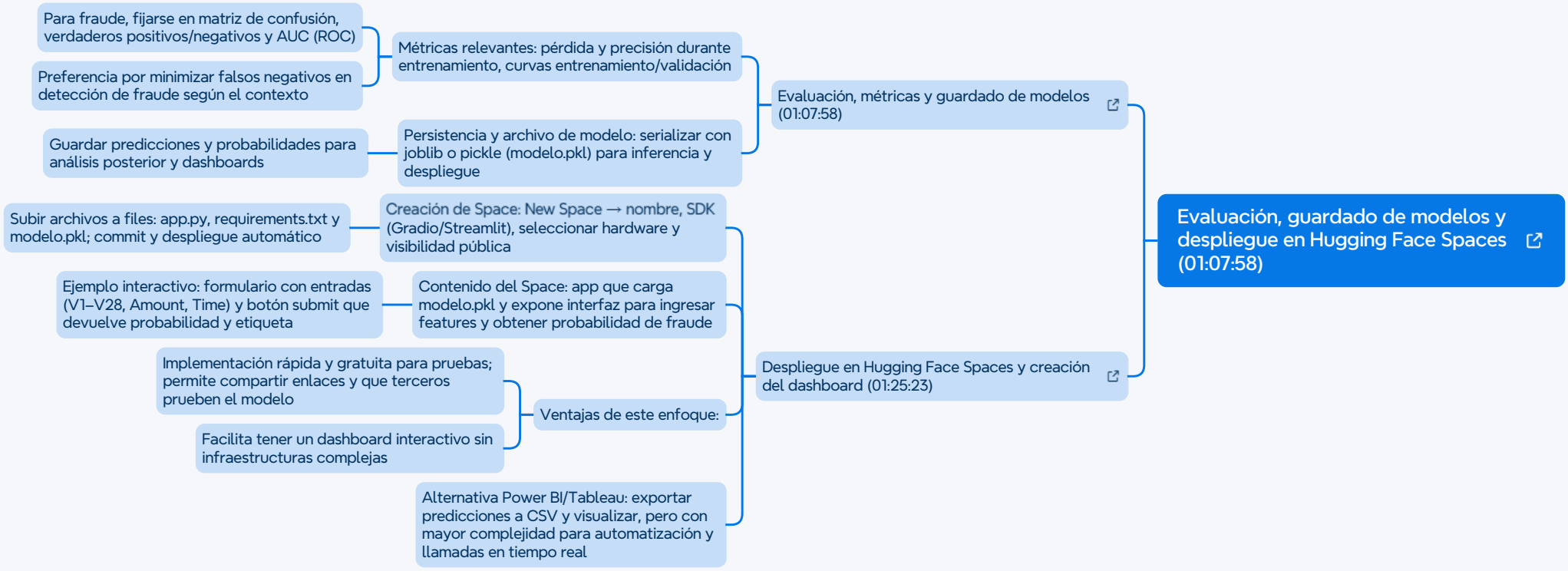

Inicio y enfoque de implementación (00:09:14)
Presentación y organización de la sesión (00:09:14)
Profesor José Chávez asume la clase y coordina pausas y tiempos
Se acuerda que el descanso (break) se definirá según el ritmo de la clase
Se pide que entren más personas y que la mayoría realice la práctica
Objetivo de la sesión: práctica completa y funcional del pipeline
Enfatiza ejecutar un ejemplo realista y utilizable en empresas
Explica razones para evitar implementaciones en la nube que requieran tarjeta o tiempo
Decisiones sobre entorno y herramientas (00:11:03)
Comparación entre implementación en nube profesional y soluciones sencillas
Plataformas: Google Cloud, Azure, AWS; costos y requisitos (tarjeta, tiempo)
Recomendación: aprender a usar nube pero presentar alternativa gratuita y práctica
Elección de herramientas gratuitas y prácticas para el pipeline
Uso de Kaggle para datos y entrenamiento en Colab con pandas
Introducción a Hugging Face: creación rápida de cuenta para integraciones y despliegues ligeros
Posibilidad de exportar a Power BI pero con limitaciones de automatización

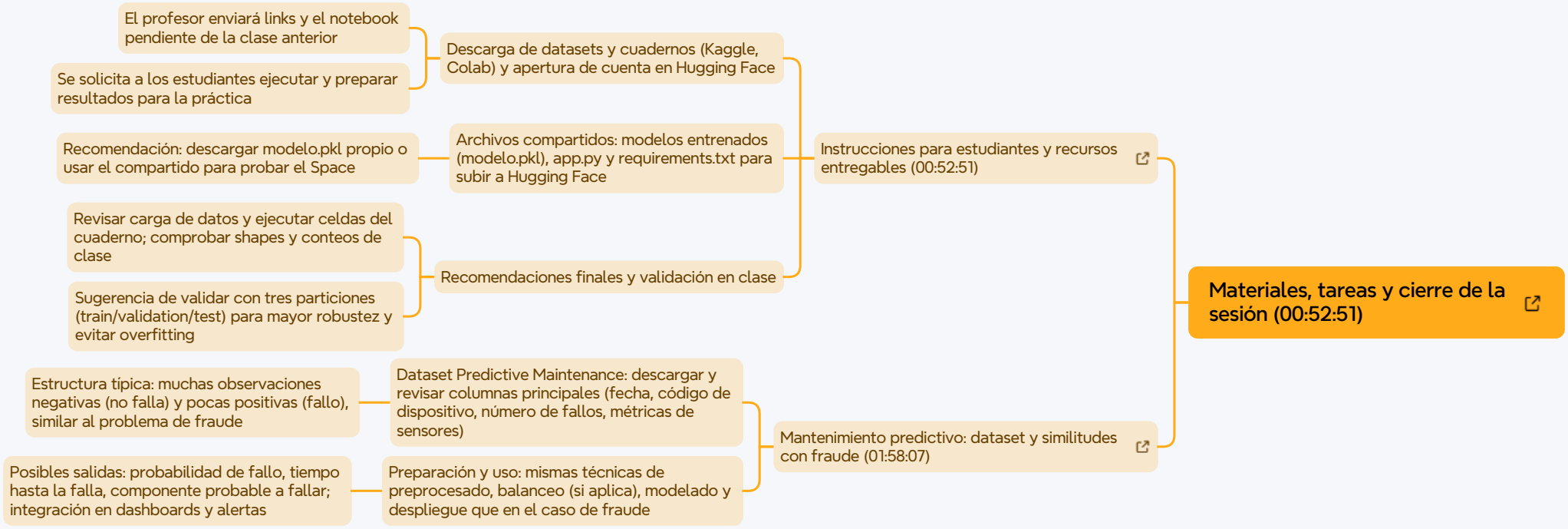

Automatización, dashboards y consideraciones de despliegue (00:14:46)
Power BI, Excel y presentación de resultados (00:14:46)
Opciones de visualización: Power BI y Excel comparadas
Power BI produce dashboards atractivos pero puede ser inflexible y poco automático
Limitaciones de Power BI: cargas manuales, integración con scripts de Python, necesidad de programación para automatizar
Enfoque práctico del curso: dashboards funcionales y directos
Prioridad en un buen modelo por encima de un dashboard estético
Se mostrarán dos enfoques: muy manual en Power BI y uno automatizado más práctico
Despliegue y orquestación (00:39:42)
Inferencia rápida y ciclo de vida del modelo
Una vez entrenado, un modelo pequeño devuelve predicciones en segundos
El modelo se usa hasta que cambie el entorno; luego se reentrena con nuevos datos
Aplicaciones donde conviene automatizar decisiones
Casos con necesidad de reactividad continua y 24/7 (líneas de producción, transporte aéreo)
Ejemplos: ajuste de velocidad de producción en tiempo real, priorización de órdenes, rutas logísticas


Casos prácticos elegidos: mantenimiento predictivo y detección de fraude (00:18:57)
Selección de datasets y motivos (00:18:57)
Elegidos: mantenimiento predictivo y detección de fraude con tarjeta de crédito
Motivo: variables numéricas, facilidad de preparación, aplicabilidad y datos reales en Kaggle
Credit Card Fraud Detection muy popular; mantenimiento predictivo relacionado con área del profesor
Preparación del entorno y materiales
Indicación para descargar datasets, abrir cuadernos Colab y crear cuenta en Hugging Face
El instructor compartirá enlaces y el notebook de Colab correspondiente
Caso práctico: predicción de velocidad óptima de producción (00:42:24)
Descripción del dataset y variables clave
Columnas: inventario, demanda proyectada, capacidad disponible, tiempo de entrega, operarios disponibles, velocidad óptima (target)
Interpretación: velocidad óptima depende de capacidad, inventario, demanda y recursos humanos
Enfoque de modelado y evaluación
Preprocesado: separar target, escalar variables, split train/test
Modelo propuesto: MLP (regresión) con ajuste de épocas y capas
Métricas: R2 como medida principal; MSE y MAE como referencias dependientes de magnitud
Ejemplos de ajuste: aumentar épocas (50→100), añadir capas densas y seleccionar variables predictivas
Relación con optimización y orquestación
El resultado puede provenir de algoritmos de optimización o heurísticas y luego alimentarse al modelo
Ciclo de vida: entrenar modelo, usar en inferencia rápida, reentrenar con nuevos datos y gestionar despliegue


Pipeline, casos y alternativas técnicas (00:20:25)
Pipeline de digitalización y ejemplos aplicados (00:20:25)
Etapas del pipeline: ingesta y captura; almacenamiento e integración; proceso y modelamiento; consulta y análisis; toma de decisiones y orquestación
El modelado (aprendizaje automático) se ubica en proceso y modelamiento
La orquestación gestiona ciclo de vida de modelos y reentrenamiento
Ejemplos sectoriales mencionados (recursos humanos, manufactura, logística, customer churn)
Similaridad conceptual entre casos; elección de ejemplos numéricos para prácticas
Alternativas a Machine Learning: algoritmos de optimización (00:22:42)
Problemas clásicos resolvibles por algoritmos (Flowshop Scheduling, shortest path, knapsack)
Ventaja: no requieren entrenamiento y son rápidos de integrar en aplicaciones
Referencia a repositorios y comunidades (Inforbix) para código y datasets
Limitaciones de algoritmos clásicos y cuándo usar ML
Algoritmos tradicionales fallan en sistemas extremadamente complejos o muy dinámicos
Métodos heurísticos y metaheurísticos ofrecen soluciones rápidas subóptimas cuando necesario

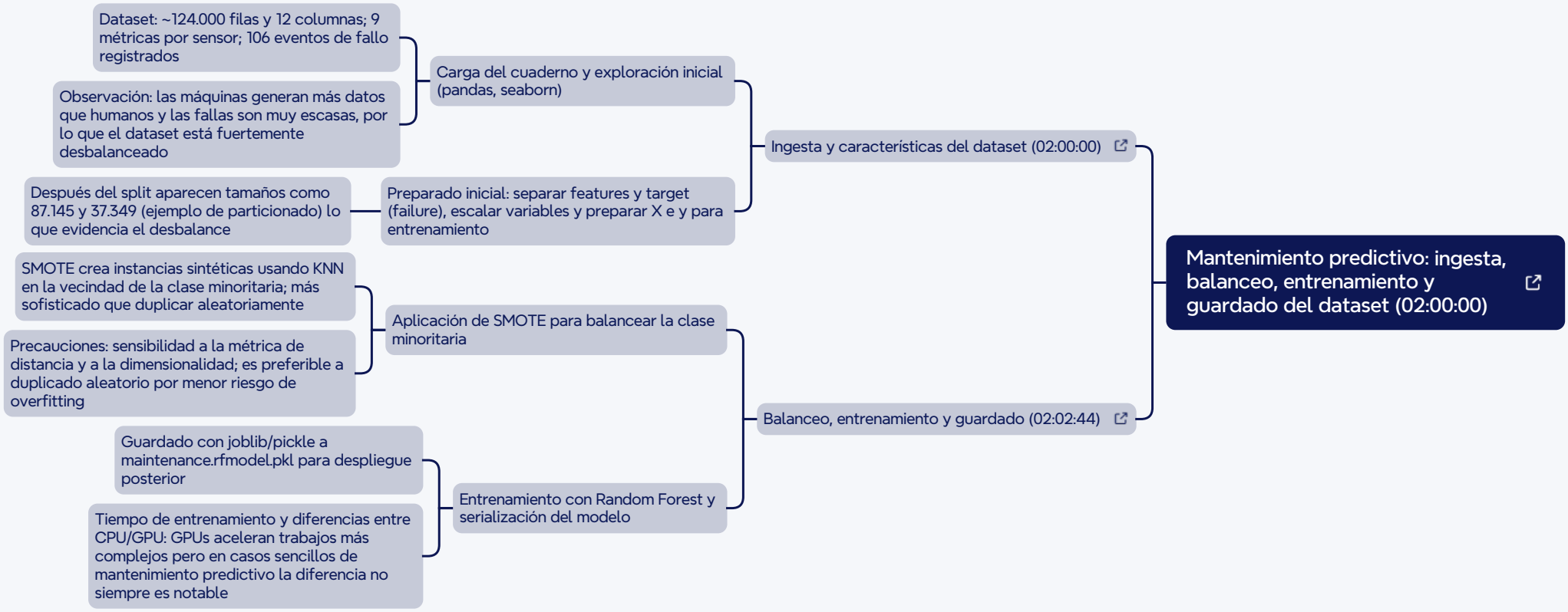

Detección de fraude: implementación práctica en Hugging Face (01:00:00)
Ingesta, exploración y características del dataset (01:00:00)
Lectura y movimientos de archivos: CSV descargado, traslado al disco de Colab y carga con pandas
Datos vienen mayormente limpios; revisar cabecera y shape para confirmar columnas
Variables y estructura del dataset Credit Card Fraud
Features V1–V28 (variables numéricas), Time (segundos desde la primera transacción), Amount (monto) y Class (0/1)
Dimensiones: 93.181 filas × 31 columnas; 92.987 no fraude y 213 fraudes (~0,17%)
Implicaciones del desbalance
Clase positiva (fraude) muy rara, lo que complica el split aleatorio y el entrenamiento
Riesgo de que el conjunto de test no contenga casos de fraude si no se maneja adecuadamente
Preprocesado, balanceo y entrenamiento (01:20:43)
Selección y limpieza de columnas: eliminar categóricas irrelevantes y variables de identificación
Opciones: codificar sectores si aportan, filtrar por sector para modelos específicos o eliminar variables con poca utilidad
Split y escalado: uso típico de train_test_split (0.2) y escalado de Amount
Atención al desbalance al hacer split; emplear random_state para reproducibilidad
Técnicas para desbalance: SMOTE y librerías (Imbalanced-Learn)
Generación de ejemplos sintéticos de la clase minoritaria para mejorar el entrenamiento
Precaución: SMOTE introduce datos simulados; usar con criterio y validar cuidadosamente
Modelos y parámetros mostrados
Keras: perceptrón multicapa para clasificación multiclase o binaria (capas densas, ReLU, softmax para multiclase)
Random Forest: ejemplo práctico con 100 árboles, random_state=42 y uso de joblib para guardar modelos (modelo.pkl)
Hiperparámetros típicos mencionados: epochs (30→50→100 o más), batch_size (16, 32), optimizador Adam

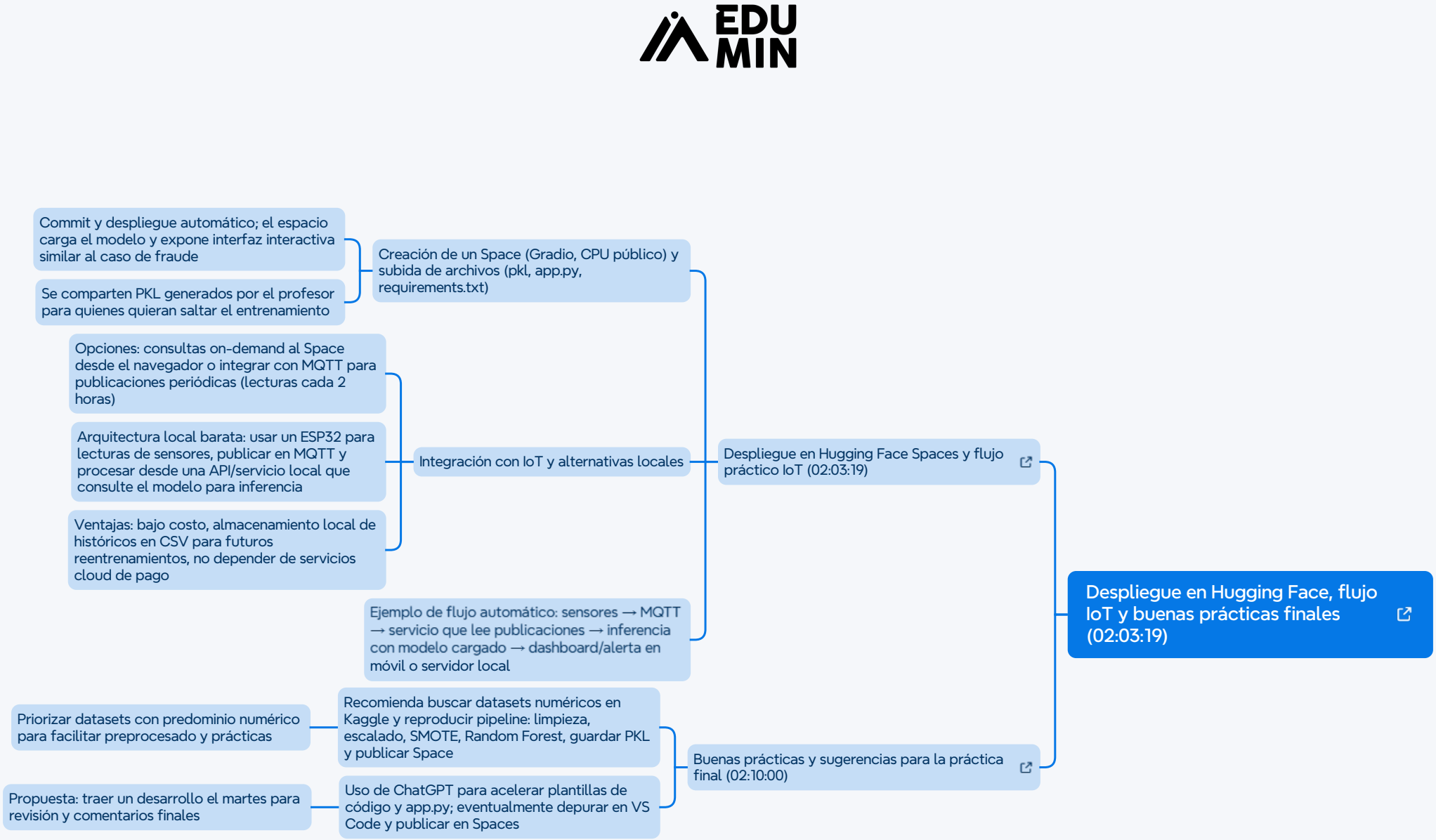

Evaluación, guardado de modelos y despliegue en Hugging Face Spaces (01:07:58)
Evaluación, métricas y guardado de modelos (01:07:58)
Métricas relevantes: pérdida y precisión durante entrenamiento, curvas entrenamiento/validación
Para fraude, fijarse en matriz de confusión, verdaderos positivos/negativos y AUC (ROC)
Preferencia por minimizar falsos negativos en detección de fraude según el contexto
Persistencia y archivo de modelo: serializar con joblib o pickle (modelo.pkl) para inferencia y despliegue
Guardar predicciones y probabilidades para análisis posterior y dashboards
Despliegue en Hugging Face Spaces y creación del dashboard (01:25:23)
Creación de Space: New Space → nombre, SDK (Gradio/Streamlit), seleccionar hardware y visibilidad pública
Subir archivos a files: app.py, requirements.txt y modelo.pkl; commit y despliegue automático
Contenido del Space: app que carga modelo.pkl y expone interfaz para ingresar features y obtener probabilidad de fraude
Ejemplo interactivo: formulario con entradas (V1–V28, Amount, Time) y botón submit que devuelve probabilidad y etiqueta
Ventajas de este enfoque:
Implementación rápida y gratuita para pruebas; permite compartir enlaces y que terceros prueben el modelo
Facilita tener un dashboard interactivo sin infraestructuras complejas
Alternativa Power BI/Tableau: exportar predicciones a CSV y visualizar, pero con mayor complejidad para automatización y llamadas en tiempo real


Materiales, tareas y cierre de la sesión (00:52:51)
Instrucciones para estudiantes y recursos entregables (00:52:51)
Descarga de datasets y cuadernos (Kaggle, Colab) y apertura de cuenta en Hugging Face
El profesor enviará links y el notebook pendiente de la clase anterior
Se solicita a los estudiantes ejecutar y preparar resultados para la práctica
Archivos compartidos: modelos entrenados (modelo.pkl), app.py y requirements.txt para subir a Hugging Face
Recomendación: descargar modelo.pkl propio o usar el compartido para probar el Space
Recomendaciones finales y validación en clase
Revisar carga de datos y ejecutar celdas del cuaderno; comprobar shapes y conteos de clase
Sugerencia de validar con tres particiones (train/validation/test) para mayor robustez y evitar overfitting
Mantenimiento predictivo: dataset y similitudes con fraude (01:58:07)
Dataset Predictive Maintenance: descargar y revisar columnas principales (fecha, código de dispositivo, número de fallos, métricas de sensores)
Estructura típica: muchas observaciones negativas (no falla) y pocas positivas (fallo), similar al problema de fraude
Preparación y uso: mismas técnicas de preprocesado, balanceo (si aplica), modelado y despliegue que en el caso de fraude
Posibles salidas: probabilidad de fallo, tiempo hasta la falla, componente probable a fallar; integración en dashboards y alertas

Ejemplo en clase: flujo práctico con Credit Card Fraud (01:11:46)

- Demostración paso a paso (01:11:46)
  - Descarga y extracción del CSV; carga en Colab; importación de librerías (pandas, seaborn, matplotlib, scikit-learn)
  - Conteo de clases y visualización: confirmar 92.987 no fraude y 213 fraudes; gráfico de barras donde la minoría casi no se nota
  - Preprocesado aplicado en la demostración: quitar Time y Class de X, normalizar Amount, hacer split 0.2 con random_state=42
  - Después balanceo con SMOTE y comprobación de tamaños iguales para ambas clases tras re-muestreo
  - Entrenamiento y guardado: entrenar Random Forest, fit, y guardar modelo.pkl; alternativa de entrenar redes Keras con softmax para multiclase
  - Resultados rápidos y tareas: observar curvas de pérdida/precisión; aumentar épocas o capas como tarea para mejorar resultados
- Creación de archivos de despliegue y uso de ChatGPT para acelerar (01:30:45)
  - Archivos necesarios para el Space: modelo.pkl, app.py y requirements.txt
  - El profesor comparte ejemplos preparados para acelerar la práctica
  - Uso de ChatGPT para generar plantillas: pedir código para app.py, requirements.txt o frontend para Hugging Face y ajustar según necesidades
  - Subida a Space y ejecución: upload files → commit → el espacio instala dependencias y ejecuta app.py automáticamente; verificar status "running"


Mantenimiento predictivo: ingesta, balanceo, entrenamiento y guardado del dataset (02:00:00)
Ingesta y características del dataset (02:00:00)
Carga del cuaderno y exploración inicial (pandas, seaborn)
Dataset: ~124.000 filas y 12 columnas; 9 métricas por sensor; 106 eventos de fallo registrados
Observación: las máquinas generan más datos que humanos y las fallas son muy escasas, por lo que el dataset está fuertemente desbalanceado
Preparado inicial: separar features y target (failure), escalar variables y preparar X e y para entrenamiento
Después del split aparecen tamaños como 87.145 y 37.349 (ejemplo de particionado) lo que evidencia el desbalance
Balanceo, entrenamiento y guardado (02:02:44)
Aplicación de SMOTE para balancear la clase minoritaria
SMOTE crea instancias sintéticas usando KNN en la vecindad de la clase minoritaria; más sofisticado que duplicar aleatoriamente
Precauciones: sensibilidad a la métrica de distancia y a la dimensionalidad; es preferible a duplicado aleatorio por menor riesgo de overfitting
Entrenamiento con Random Forest y serialización del modelo
Guardado con joblib/pickle a maintenance.rfmodel.pkl para despliegue posterior
Tiempo de entrenamiento y diferencias entre CPU/GPU: GPUs aceleran trabajos más complejos pero en casos sencillos de mantenimiento predictivo la diferencia no siempre es notable


Despliegue en Hugging Face, flujo IoT y buenas prácticas finales (02:03:19)
Despliegue en Hugging Face Spaces y flujo práctico IoT (02:03:19)
Creación de un Space (Gradio, CPU público) y subida de archivos (pkl, app.py, requirements.txt)
Commit y despliegue automático; el espacio carga el modelo y expone interfaz interactiva similar al caso de fraude
Se comparten PKL generados por el profesor para quienes quieran saltar el entrenamiento
Integración con IoT y alternativas locales
Opciones: consultas on-demand al Space desde el navegador o integrar con MQTT para publicaciones periódicas (lecturas cada 2 horas)
Arquitectura local barata: usar un ESP32 para lecturas de sensores, publicar en MQTT y procesar desde una API/servicio local que consulte el modelo para inferencia
Ventajas: bajo costo, almacenamiento local de históricos en CSV para futuros reentrenamientos, no depender de servicios cloud de pago
Ejemplo de flujo automático: sensores → MQTT → servicio que lee publicaciones → inferencia con modelo cargado → dashboard/alerta en móvil o servidor local
Buenas prácticas y sugerencias para la práctica final (02:10:00)
Recomienda buscar datasets numéricos en Kaggle y reproducir pipeline: limpieza, escalado, SMOTE, Random Forest, guardar PKL y publicar Space
Priorizar datasets con predominio numérico para facilitar preprocesado y prácticas
Uso de ChatGPT para acelerar plantillas de código y app.py; eventualmente depurar en VS Code y publicar en Spaces
Propuesta: traer un desarrollo el martes para revisión y comentarios finales

EDU MIN

Conclusiones prácticas y recomendaciones (01:39:19)

- Principales aprendizajes prácticos
  - Con herramientas gratuitas (Colab, Kaggle, Hugging Face) se puede implementar un pipeline completo en poco tiempo
  - Desde ingestión hasta dashboard interactivo y despliegue ligero sin infraestructura costosa
  - Importancia del preprocesado, manejo de desbalance y validación para obtener modelos útiles en producción
- Buenas prácticas sugeridas
  - Priorizar modelos reproducibles y guardados (modelo.pkl) para facilitar deployment y pruebas
  - Usar joblib/pickle, incluir requirements.txt y una app mínima (app.py) para exponer el servicio
  - Elegir la visualización según el caso: Hugging Face/Gradio para prototipos interactivos; Power BI/Tableau para informes, con la salvedad de mayor complejidad para automatizar
  - Documentar decisiones (features usadas, tratamiento de categóricas, balanceo) y dejar tareas claras para mejorar el modelo tras la clase